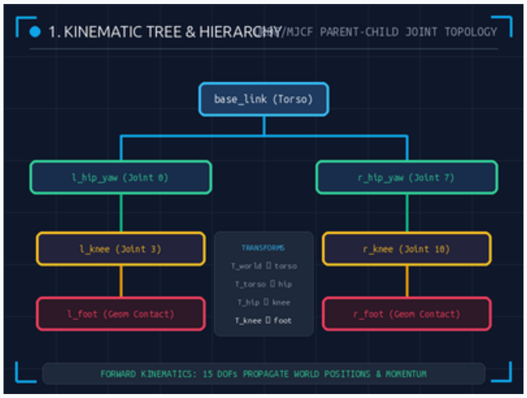
■ MICRODUCK PHYSICAL AI MASTERCLASS

PHASE 2 HANDOUT

Module 2: The Invisible Matrix: Kinematic Trees, Collision Geoms, Forward Dynamics & MJCF

MuJoCo Physics

ENGINE: MuJoCo 3.x	INTEGRATOR: mj_step() @ 50Hz	TREE: 15 Joints DAG	COMPILER: autolimits='true'



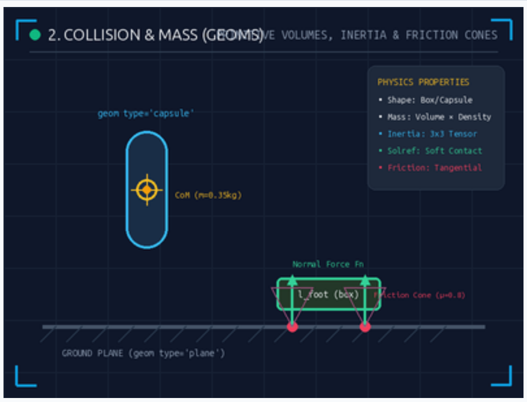
■ 1. Kinematic Trees & DAG Topology

PARENT-CHILD COORDINATE FRAMES

Hierarchical Transforms: Robot bodies are structured as Directed Acyclic Graphs (DAGs). Rotations at parent joints (hip) automatically propagate through child links (knee, foot).

The 12-Year-Old Analogy: When you swing your shoulder, your elbow and hand move along automatically because they are attached children in the tree.

- **Base Link Origin**: Root floating base (base_link) has 6 unactuated DOFs (position + quaternion).
- **Joint Articulations**: 15 1-DOF revolute joints defined in radians relative to parent link frames.
- **End Effectors**: Feet contact positions computed via recursive forward kinematics chain.



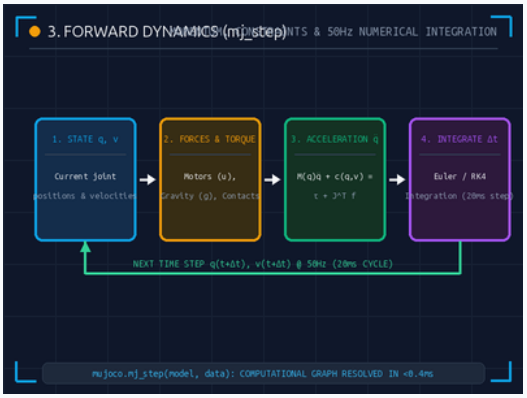
■ 2. Collision & Mass (Geoms)

BOUNDING VOLUMES & CONTACT SOLVERS

Fast Collision Primitives: Physics engines do not collide 50k-triangle visual meshes. Instead, Geoms encapsulate bodies in convex primitives (capsules, boxes, spheres).

The 12-Year-Old Analogy: Instead of calculating every feather on a duck, the computer wraps the foot in a smooth rubber box that bounces cleanly off the floor.

- **Contact Dynamics**: Soft constraint solver (solref) avoids numerical stiffness spikes on foot strike.
- **Friction Cones**: Tangential friction $\mu=0.8$ prevents foot slippage during dynamic walking push-off.
- **Inertia Tensors**: 3×3 rotational inertia matrices derived from primitive bounding volumes.



3. Forward Dynamics: mj_step()

EQUATIONS OF MOTION & 50HZ LOOP

Equations of Motion: Resolves $M(q)\ddot{q} + c(q,v) = \tau + J^T f$ to calculate joint accelerations $\ddot{q}$ from motor torques, gravity, and ground reaction forces.

The 12-Year-Old Analogy: A math engine solving 50 physics puzzles a second to calculate where momentum throws the duck after each step.

- **50Hz Heartbeat**: Executes with fixed $\Delta t = 0.020\text{s}$ cadence matching real robotd kernel.
- **State Evolution**: Integrates accelerations to update joint velocities $v(t+\Delta t)$ and positions $q(t+\Delta t)$.
- **Computational Efficiency**: MuJoCo executes full 15-DOF dynamics in <0.35ms per step.

4. MJCF Wrapper Architecture

DYNAMIC URDF IMPORT & MOTOR INJECTION

The Wrapper Pattern: Vendor URDFs lack motor definitions and physics tuning. The MJCF Wrapper dynamically imports the URDF and injects 15 actuators and safety bounds.

The 12-Year-Old Analogy: Downloading a toy car 3D model and dropping a real electric motor and battery inside its chassis.

- **autolimits='true'**: Infers missing link inertias from mesh volumes, preventing zero-mass divide-by-zero crashes.
- **Actuator Injection**: Adds 15 <motor> tags bounded with ctrlrange='-1.0 1.0'.
- **Unified Compilation**: mujoco.mj_saveLastXML() dumps merged, fully optimized model to disk.

■ PHASE 2 PHYSICS SUMMARY & REINFORCEMENT LEARNING HANDOFF

- **MjModel vs MjData:** Static blueprint defines constraints; live state tracks positions and momentum.
- **Hardware Safety:** Geoms and solref parameters ensure accurate ground contacts without simulation exploding.
- **Ready for Training:** With the virtual duck compiled, proceed to Phase 3: The Dog Trainer (PPO).

■ PHASE 2 KNOWLEDGE CHECK

ASSESSMENT

Test your understanding of the MuJoCo physics matrix before training reinforcement learning policies.

5 Questions

Q1: Physics Blueprint vs Live State

MULTIPLE CHOICE

What is the key operational difference between MjModel and MjData?

- **A.** MjModel changes dynamically while MjData is frozen.
- **B.** MjModel is the static physics blueprint; MjData holds the live state (qpos, qvel).
- **C.** MjData only stores camera images.

Q2: Collision Geometry Efficiency

MULTIPLE CHOICE

Why do physics engines use Geoms (capsules/boxes) instead of high-poly 3D meshes for contacts?

- **A.** Bounding shapes make contact and friction math over 100x faster.
- **B.** Meshes cannot have color in MuJoCo.
- **C.** Geoms prevent gravity from affecting the robot.

Q3: Forward Dynamics Step

MULTIPLE CHOICE

What occurs when calling mujoco.mj_step(model, data)?

- **A.** It reboots the Linux operating system.
- **B.** It calculates forces, accelerations, and integrates time forward by one Δt step.
- **C.** It automatically converts Python code to C++.

Q4: The Wrapper Pattern

MULTIPLE CHOICE

Why do we use an MJCF Wrapper instead of directly editing the vendor URDF?

- **A.** It injects 15 motors and physics rules without modifying the vendor's source file.
- **B.** URDF files cannot be read by Python.
- **C.** It reduces the robot's physical weight by 50%.

Q5: Autolimits & Mass Singularities

MULTIPLE CHOICE

What happens if autolimits='true' is omitted when a vendor link has mass=0.0?

- **A.** The robot moves normally.
- **B.** Division by zero occurs and the simulation explodes with NaN velocity.
- **C.** The battery recharges instantly.

Answer Key: 1-B, 2-A, 3-B, 4-A, 5-B (1-B: Blueprint vs State; 2-A: Contact speed; 3-B: Force integration; 4-A: Motor injection; 5-B: Zero-mass NaN crash)